

Российский университет дружбы народов

Факультет физико-математических и естественных наук

Сравнительный анализ алгоритмов активного управления очередями: RED, ARED, Gentle RED

Выполнил:

Палагашвили Абули Михайлович

аспирант

ORCID: 0009-0007-7009-7038

E-mail: 1042250185@rudn.ru

Содержание

1	Введение	2
1.1	Актуальность исследования	2
1.2	Обзор литературы	2
1.3	Цель и задачи исследования	3
1.4	Научная новизна	4
1.5	Алгоритмы семейства RED	4
1.5.1	RED (Random Early Detection)	5
1.5.2	ARED (Adaptive RED)	5
1.5.3	Gentle RED	5
2	Методы	6
2.1	Топология эксперимента	6
2.2	Конфигурация AQM	7
2.3	Инструментарий	7
2.4	Метрики оценки	8
3	Результаты	8
3.1	Динамика очереди	8
3.2	Пропускная способность	10
3.3	Сбросы пакетов	12
4	Обсуждение	13
5	Заключение	14

1 Введение

1.1 Актуальность исследования

Непрерывный рост объёмов интернет-трафика, обусловленный развитием облачных вычислений, потокового видео и сетей 5G/6G, предъявляет всё более жёсткие требования к механизмам управления перегрузкой в сетевых узлах. Традиционный подход — дроп хвоста (tail-drop), при котором пакеты отбрасываются только при полном заполнении буфера, — приводит к глобальной синхронизации TCP-потокa, резким осцилляциям пропускной способности и эффекту bufferbloat [5]. Проблема bufferbloat — систематическое переполнение буферов в маршрутизаторах, вызывающее рост сквозной задержки на порядки, — была идентифицирована как одна из ключевых угроз качеству обслуживания [7].

Алгоритмы активного управления очередями (AQM, Active Queue Management), рекомендованные IETF в RFC 2309 [6], решают эту проблему превентивным отбрасыванием или маркировкой пакетов до переполнения буфера. Семейство алгоритмов RED (Random Early Detection) остаётся наиболее широко развёрнутым классом AQM-механизмов: RED реализован во всех основных сетевых операционных системах (Cisco IOS, Linux tc, FreeBSD ALTQ), а его варианты — ARED и Gentle RED — поставляются как стандартные модули в симуляторе ns-3 и в ядре Linux. Несмотря на появление новых алгоритмов (CoDel [8], PIE [9]), выбор конкретного варианта RED для конкретных условий эксплуатации по-прежнему требует экспериментального обоснования, поскольку аналитические модели не учитывают всех аспектов взаимодействия с гетерогенным TCP-трафиком.

1.2 Обзор литературы

Алгоритм RED был предложен Флойдом и Якобсоном в 1993 году [1] и стал первым практическим механизмом AQM. Авторы продемонстрировали, что вероятностное раннее отбрасывание пакетов на основе экспоненциально взвешенного скользящего среднего (EWMA) длины очереди позволяет избежать глобальной синхронизации TCP-потокa и снизить среднюю задержку в буфере по сравнению с tail-drop.

Практическое развёртывание RED выявило его главный недостаток — высокую чувствительность к настройке параметров. Кристиансен и др. [4] показали, что производительность RED существенно зависит от соотношения $\min_{Th}$, $\max_{Th}$ и $\max_P$, а оптимальные значения меняются при изменении числа потокa и уровня нагрузки. Для преодоления этого ограничения Флойд и др. [3] предложили ARED (Adaptive RED), в котором параметр $\max_P$ автоматически адаптируется к текущей нагрузке на основе отклонения средней очереди от целевого диапазона.

Параллельно Флойд [2] предложила модификацию Gentle RED, устраняющую резкий переход вероятности сброса от $\max_P$ к 1 при $\bar{q} = \max_{Th}$. Gentle RED вводит дополнительную линейную зону $[\max_{Th}, 2 \cdot \max_{Th})$, в которой вероятность плавно возрастает

до единицы, что снижает синхронизацию TCP-потоков и уменьшает дисперсию задержки при кратковременных всплесках.

Значительный вклад в математическое моделирование взаимодействия TCP и RED внесла Королькова А.В. [12], построившая математическую модель процесса передачи трафика с регулируемой алгоритмом типа RED динамической интенсивностью потока. В её работе с помощью аппарата стохастических дифференциальных уравнений с пуассоновским процессом впервые была формализована автономная система трёх обыкновенных дифференциальных уравнений, описывающих совместную динамику размера TCP-окна, мгновенной длины очереди и EWMA-средней. Был предложен метод определения области возникновения автоколебательного режима, а также проведён качественный анализ модели с использованием алгоритмов RED, GRED, DSRED и ARED в симуляторе NS-2, подтвердивший наличие двух устойчивых положений равновесия (узла и предельного цикла), определяющих отсутствие или наличие автоколебаний.

Развитие этого направления было продолжено Велиевой Т.Р. [13], исследовавшей нелинейные явления и границы области возникновения автоколебательного режима в непрерывно-дискретных динамических моделях. Ключевым вкладом этой работы стала адаптация метода Крылова–Боголюбова и метода гармонической линеаризации к непрерывно-дискретной модели TCP/RED, что позволило аналитически определить амплитудно-частотные характеристики автоколебаний и границы области их возникновения в пространстве параметров. Результаты аналитического исследования были верифицированы сравнением с имитационным моделированием в ns-2.

Среди альтернативных подходов к AQM следует отметить алгоритм BLUE [10], управляющий вероятностью сброса на основе событий потерь и простоя канала, а также семейство алгоритмов на основе теории управления — PI-контроллер Холлота и др. [11] и его развитие PIE [9]. Алгоритм CoDel [8] использует принципиально иной подход, отслеживая время пребывания пакета в очереди (sojourn time) вместо длины очереди.

Таким образом, несмотря на обширные исследования отдельных аспектов алгоритмов семейства RED — как аналитических (устойчивость, автоколебания), так и экспериментальных, — систематическое трёхстороннее сравнение RED, ARED и Gentle RED при строго идентичных условиях с использованием расширенного набора метрик (включая временную структуру сбросов и зональное распределение очереди) в литературе представлено недостаточно.

1.3 Цель и задачи исследования

Цель работы — провести сравнительный анализ трёх алгоритмов семейства RED (RED, ARED, Gentle RED) в условиях устойчивой перегрузки сетевого узла и выявить количественные различия в динамике управления буфером, стабильности пропускной способности и характере сигналов о перегрузке.

Для достижения цели поставлены следующие **задачи**:

1. Реализовать симуляционную модель сети с топологией «гантель» в среде ns-3 с возможностью переключения алгоритма AQM при сохранении прочих условий.
2. Провести серию экспериментов при идентичных параметрах очереди и нагрузки для каждого из трёх алгоритмов.
3. Сравнить алгоритмы по комплексу метрик: средняя и хвостовая длина очереди, коэффициент вариации пропускной способности, число и временная структура сбросов пакетов.
4. Сформулировать практические рекомендации по выбору алгоритма AQM в зависимости от приоритетных требований сценария.

1.4 Научная новизна

Научная новизна работы состоит в следующем:

1. В отличие от работ Флойда и Якобсона [1], Флойда [2,3] и Кристиансена и др. [4], где каждый алгоритм анализировался изолированно или в парном сравнении с tail-drop, в данной работе проведено систематическое трёхстороннее сравнение RED, ARED и Gentle RED при строго идентичных условиях (единая топология, одинаковые пороги, одинаковый профиль нагрузки), что исключает влияние различий в методологии на результаты.
2. В отличие от аналитического подхода Корольковой [12], исследовавшей автоколебательные режимы с помощью систем ОДУ, и Велиевой [13], применявшей метод гармонической линеаризации для определения границ устойчивости, в данной работе предложен расширенный набор *экспериментальных* метрик оценки, включающий зональное распределение времени работы EWMA-средней и временную структуру сбросов (медиана интервала, кластерность), что позволяет охарактеризовать не только статические показатели, но и динамику управляющих воздействий в имитационной модели.
3. В отличие от работ Холлота и др. [11] и Фэна и др. [10], сосредоточенных на сравнении RED с принципиально иными подходами к AQM (PI-контроллер, BLUE), данная работа фокусируется на *внутрисемейственном* сравнении вариантов RED и впервые количественно описывает фундаментальный компромисс между предсказуемостью пропускной способности (RED, CV = 18.7%), минимизацией сбросов (ARED, -19%) и минимизацией задержки в буфере (Gentle RED, средняя очередь 6.40 пкт).

1.5 Алгоритмы семейства RED

Алгоритмы активного управления очередями начинают отбрасывать или помечать пакеты *заблаговременно* — до того, как буфер переполнится — на основании статистики

средней длины очереди. TCP-соединения получают сигнал о перегрузке раньше и плавнее снижают скорость отправки, не доводя ситуацию до коллапса.

1.5.1 RED (Random Early Detection)

RED — базовый алгоритм AQM, предложенный Флойдом и Якобсоном в 1993 году [1]. Алгоритм вычисляет экспоненциально взвешенное скользящее среднее длины очереди (EWMA) и принимает решение о сбросе на его основе.

Обновление средней очереди при каждом поступающем пакете:

$$\bar{q} = (1 - w_q) \bar{q} + w_q q \quad (1)$$

где q — текущая (мгновенная) длина очереди, $w_q \in (0, 1)$ — вес фильтра.

Вероятность сброса определяется тремя зонами:

$$p(\bar{q}) = \begin{cases} 0 & \bar{q} < \min_{Th} \\ \max P \cdot \frac{\bar{q} - \min_{Th}}{\max_{Th} - \min_{Th}} & \min_{Th} \leq \bar{q} < \max_{Th} \\ 1 & \bar{q} \geq \max_{Th} \end{cases} \quad (2)$$

Жёсткий принудительный сброс выше $\max_{Th}$ обеспечивает быстрое реагирование, но синхронизирует TCP-потоки и создаёт пилообразные осцилляции очереди.

1.5.2 ARED (Adaptive RED)

ARED расширяет RED адаптивным механизмом [3]: параметр $\max P$ периодически пересчитывается через каждые $T_{interval}$ в зависимости от отклонения средней очереди от целевого диапазона:

$$\max P = \begin{cases} \max P + \alpha & \bar{q} > \frac{\min_{Th} + \max_{Th}}{2} \\ \max P - \beta & \bar{q} < \frac{\min_{Th} + \max_{Th}}{2} \end{cases} \quad (3)$$

где α и β — коэффициенты увеличения и снижения вероятности ($\alpha > \beta$). Значение $\max P$ ограничивается допустимым диапазоном. Это позволяет алгоритму приспосабливаться к меняющейся нагрузке без ручной настройки, однако адаптация вносит дополнительную инерцию и может приводить к широким колебаниям очереди.

1.5.3 Gentle RED

Gentle RED [2] модифицирует поведение RED в зоне выше $\max_{Th}$: вместо немедленного принудительного сброса алгоритм продолжает вероятностный подход вплоть до

$2 \cdot \max_{Th}$:

$$p(\bar{q}) = \begin{cases} 0 & \bar{q} < \min_{Th} \\ \max P \cdot \frac{\bar{q} - \min_{Th}}{\max_{Th} - \min_{Th}} & \min_{Th} \leq \bar{q} < \max_{Th} \\ \max P + (1 - \max P) \cdot \frac{\bar{q} - \max_{Th}}{\max_{Th}} & \max_{Th} \leq \bar{q} < 2 \max_{Th} \\ 1 & \bar{q} \geq 2 \max_{Th} \end{cases} \quad (4)$$

В зоне $[\max_{Th}, 2 \cdot \max_{Th})$ вероятность линейно растёт от $\max P$ до 1, а принудительный сброс происходит лишь при $\bar{q} \geq 2 \cdot \max_{Th}$. Это смягчает реакцию на кратковременные всплески, снижает синхронизацию ТСП-потоков и уменьшает среднюю длину очереди — ценой несколько большей вариабельности пропускной способности.

2 Методы

2.1 Топология эксперимента

В эксперименте используется топология типа «гантель» (dumbbell): несколько отправителей подключены к общему маршрутизатору через быстрые каналы, а маршрутизатор соединён с единственным получателем через узкий канал с ограниченной пропускной способностью. Узкий канал является единственным местом возможной перегрузки, и именно на его исходящем интерфейсе устанавливается дисциплина очереди AQM.

```

Sender 0  --\
Sender 1  --|
Sender 2  --|  1 Mbit/s          2 Mbit/s
...      |-----> [Router] -----> Receiver
Sender 8  --|  delay 2 ms        delay 10 ms
Sender 9  --/
                ^
                AQM queue
                (RED / ARED / Gentle)

```

Каждый из десяти отправителей устанавливает одно ТСП-соединение с получателем и непрерывно передаёт данные (BulkSend). Суммарная входящая нагрузка на маршрутизатор потенциально достигает $10 \times 1 \text{ Мбит/с} = 10 \text{ Мбит/с}$, что в 5 раз превышает пропускную способность узкого канала. Это обеспечивает устойчивую перегрузку и позволяет оценить поведение алгоритмов AQM в реальных условиях насыщения буфера.

Параметры топологии приведены в табл. 1.

Таблица 1: Параметры топологии и симуляции

Параметр	Значение	Описание
nSenders	10	Количество TCP-отправителей
fastRate	1 Мбит/с	Скорость каналов отправитель → маршрутизатор (задержка)
bottleneckRate	2 Мбит/с	Скорость узкого канала маршрутизатор → получатель
bottleneckDelay	10 мс	Задержка узкого канала
simTime	60.0	Длительность симуляции (с)
Транспорт	TCP BulkSend	Размер сегмента 512 байт

2.2 Конфигурация AQM

Все три алгоритма запускаются с одинаковыми пороговыми параметрами (табл. 2).

Таблица 2: Единые пороговые параметры для RED, ARED и Gentle RED

Параметр	Значение	Описание
minTh	5 пакетов	Нижний порог средней очереди — ниже него сбросов нет
maxTh	10 пакетов	Верхний порог; выше него RED/ARED сбрасывают все пакеты
MaxSize	100 пакетов	Жёсткий лимит буфера

2.3 Инструментарий

В работе использовался дискретно-событийный симулятор сетей **ns-3** версии 3.41. Симуляционные скрипты размещаются в директории **scratch/**. Для данного эксперимента создана поддиректория со следующей структурой:

```
ns-3.41/
+-- scratch/
    +-- red-experiments/
        +-- red.cc
        +-- configs/
            | +-- red.txt
            | +-- ared.txt
            | +-- gentle.txt
        +-- analysis/
            +-- vis.py
            +-- ...
```

Конфигурационные файлы задают тип AQM и параметры очереди. Пример `configs/red.txt`

```
aqmType=red
minTh=5
maxTh=10
MaxSize=100
```


Аналогичные файлы для ARED (aqmType=ared) и Gentle RED (aqmType=gentle) отличаются лишь значением поля aqmType.

Симуляции запускались командой:

```
./ns3 run "scratch/red-experiments/red.cc --config=configs/<algorithm>.txt"
```

По завершении каждой симуляции три CSV-файла перемещались в директорию analysis/:

- <algorithm>_queue.csv — длина очереди раз в 100 мс;
- <algorithm>_drops.csv — метки времени каждого сброса пакета;
- <algorithm>_throughput.csv — пропускная способность раз в 100 мс.

Для построения графиков использовался скрипт vis.py со сглаживанием скользящим средним (окно 30 отсчётов):

```
python vis.py --algorithms red ared gentle --window 30
```

2.4 Метрики оценки

Таблица 3: Метрики оценки алгоритмов

Метрика	Источник данных	Что характеризует
Средняя и медианная длина очереди	*_queue.csv	Средняя задержка в буфере
p90 / p99 длины очереди	*_queue.csv	Хвостовые задержки
Доля времени с пустой очередью	*_queue.csv	Недоиспользование канала
Средняя пропускная способность	*_throughput.csv	Утилизация канала
Коэффициент вариации (CV)	*_throughput.csv	Стабильность потока
Общее число сбросов	*_drops.csv	Нагрузка на TCP-механизм перегрузки
Медиана интервала между сбросами	*_drops.csv	Характер управляющих сигналов

3 Результаты

Эксперимент состоял из трёх последовательных симуляций — по одной на каждый алгоритм. Каждый из трёх алгоритмов запускался с одинаковыми параметрами очереди (табл. 2) в течение 60 секунд (первая секунда — прогрев, из анализа исключена).

3.1 Динамика очереди

На рис. 1 представлена динамика длины очереди для трёх алгоритмов за всё время симуляции.

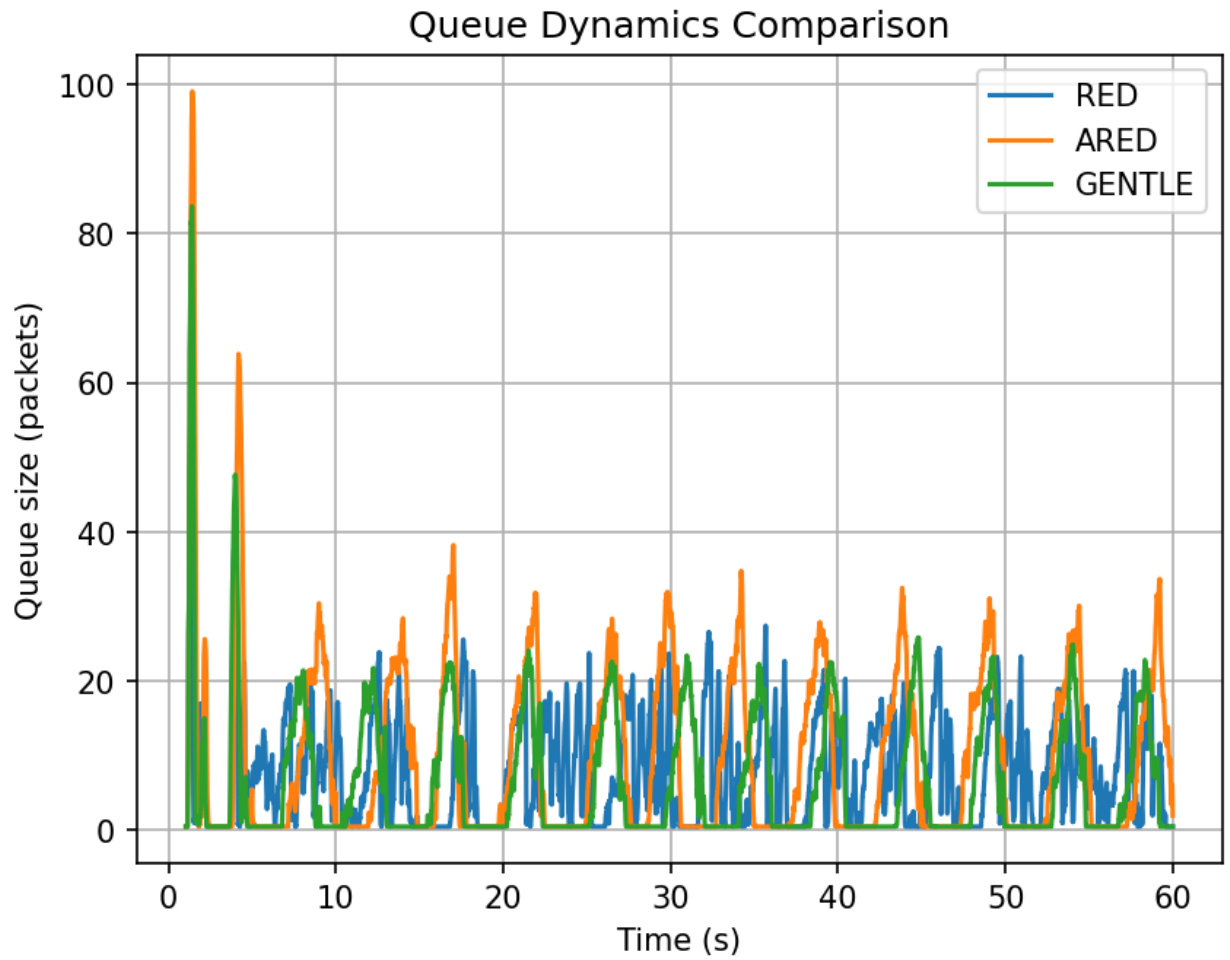


Рис. 1: Динамика длины очереди для RED, ARED и Gentle RED

На графике чётко выделяются две фазы.

Фаза медленного старта ($t = 0-4$ с). Все TCP-соединения одновременно входят в slow-start и за секунды насыщают буфер. Очередь у всех алгоритмов достигает пиковых значений: ARED — до 100 пакетов, RED и Gentle RED — до 84 пакетов. Этот выброс одинаков для всех трёх, поскольку алгоритм AQM ещё не успел накопить статистику и среагировать.

Установившийся режим ($t > 5$ с). После рассасывания начального всплеска алгоритмы расходятся в поведении.

RED (синяя кривая) удерживает очередь в диапазоне 5–26 пакетов с относительно регулярными осцилляциями. Кривая редко касается нуля (пустая очередь лишь 13.4% времени): принудительный сброс при $m_qAvg \geq maxTh$ реагирует быстро и держит очередь «в узде», но ценой постоянного давления на TCP-потoki.

ARED (оранжевая кривая) демонстрирует наибольшую амплитуду колебаний и наиболее хаотичный рисунок. Периодически очередь вырастает до 30–35 пакетов в установившемся режиме и резко обваливается к нулю. Это следствие адаптации $maxP$: алгоритм поочерёдно «отпускает» и «закручивает» вероятность сброса, порождая широкие и нерегулярные циклы. $p99 = 49$ пакетов — самое высокое значение среди трёх алгоритмов.

Gentle RED (зелёная кривая) чаще остальных возвращается к нулю (26.6% времени очередь пуста, медиана = 1 пакет). Плавная вероятностная функция в зоне $[\max_{Th}, 2 \cdot \max_{Th}]$ позволяет ТСП-потокам десинхронизировать свои окна, что приводит к более глубоким, но менее длительным «провалам» очереди.

Статистика установившегося режима приведена в табл. 4.

Таблица 4: Статистика длины очереди в установившемся режиме

Алгоритм	Среднее (пкт)	Стд. откл.	Медиана	p90	p99	Пустая (%)
RED	7.74	7.45	6	18	26	13.4%
ARED	9.57	11.94	4	25	49	22.3%
Gentle	6.40	8.86	1	19	34	26.6%

Распределение времени по зонам работы EWMA-средней $\bar{q}$ показано в табл. 5.

Таблица 5: Доля времени в каждой зоне управления очередью

Алгоритм	$\bar{q} < \min_{Th}$	$[\min_{Th}, \max_{Th})$	$[\max_{Th}, 2\max_{Th})$	$\bar{q} \geq 2\max_{Th}$
RED	46.0%	19.8%	29.3%	4.8%
ARED	52.6%	8.0%	19.6%	19.7%
Gentle	60.8%	11.8%	19.8%	7.7%

RED проводит 34.1% времени в зоне принудительного сброса ($m_{qAvg} \geq \max_{Th}$), что объясняет его высокую среднюю очередь. ARED проводит 19.7% времени в зоне $\bar{q} \geq 2 \cdot \max_{Th}$, где дропаются все пакеты, — отсюда его аномально высокое p99. Gentle RED чаще всего находится ниже $\min_{Th}$ (60.8%) и реже всего попадает в зону жёстких сбросов (7.7%).

3.2 Пропускная способность

График пропускной способности (рис. 2) построен с применением скользящего среднего с окном 30 отсчётов, поэтому отображает данные начиная с $t \approx 29$ с. Ось Y намеренно сжата до диапазона 1.74–1.86 Мбит/с, что позволяет рассмотреть различия, невидимые на полной шкале.

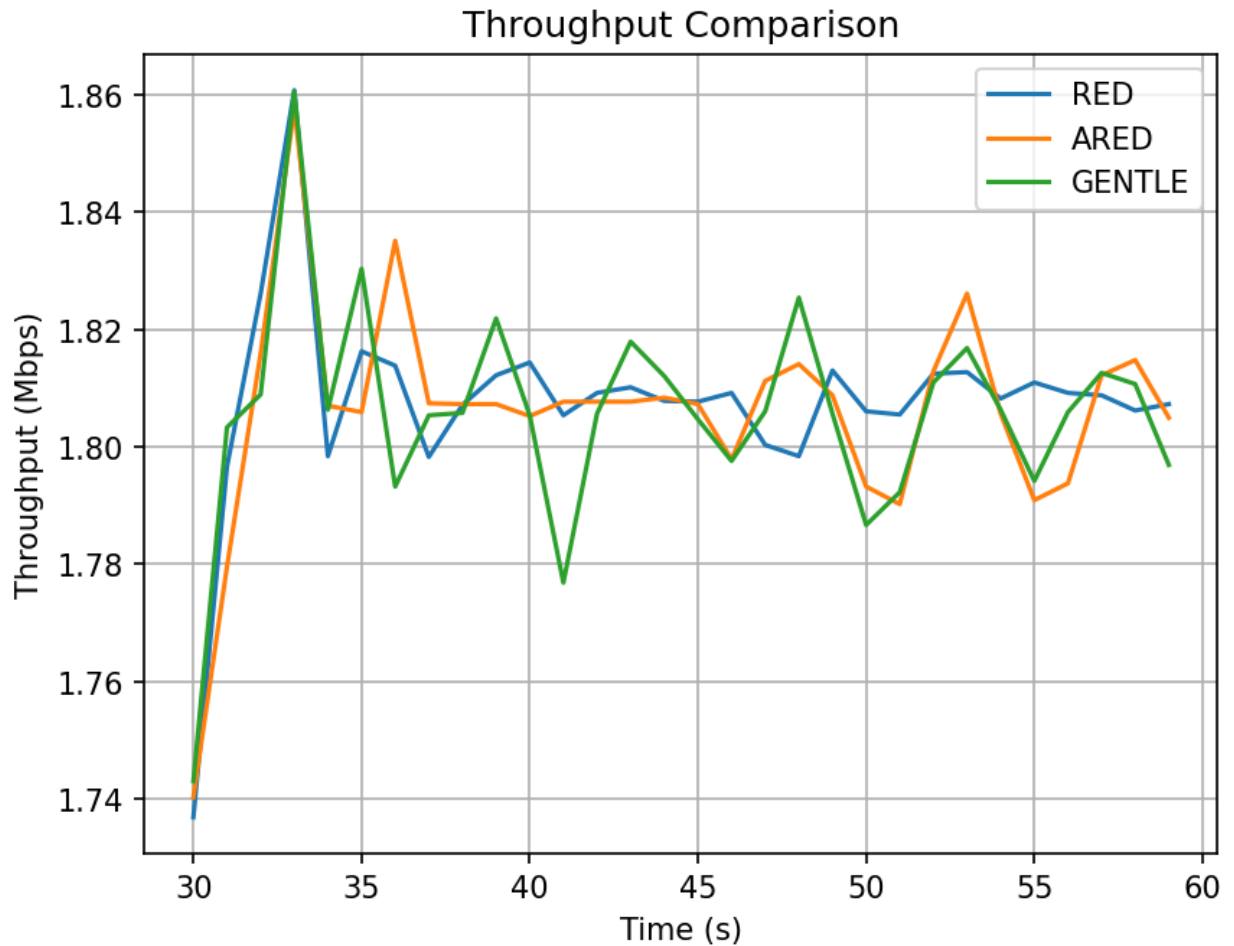


Рис. 2: Пропускная способность: сравнение RED, ARED и Gentle RED

Все три алгоритма стабильно удерживают пропускную способность вблизи **1.80–1.81 Мбит/с** ($\sim 90\%$ от номинала канала 2 Мбит/с). Это означает, что выбор алгоритма AQM при данных параметрах не влияет на утилизацию полосы: все три механизма одинаково эффективно «заполняют» канал.

Характерный пик около $t = 33$ с (все кривые достигают ~ 1.86 Мбит/с) — синхронное восстановление TCP-потоков после первоначальной волны сбросов: окна конгестии одновременно начинают расти, создавая кратковременный избыточный поток.

Таблица 6: Статистика пропускной способности (CV — коэффициент вариации)

Алгоритм	Среднее (Мбит/с)	Стд. откл.	Мин.	Макс.	CV
RED	1.8047	0.3380	0.533	3.621	18.7%
ARED	1.8039	0.3950	0.553	3.293	21.9%
Gentle	1.7998	0.4444	0.352	3.654	24.7%

Несмотря на одинаковое среднее, **RED демонстрирует наименьшую вариативность** ($CV = 18.7\%$): жёсткий синхронный сброс держит все TCP-потоки в такт, делая пропускную способность предсказуемой. Gentle RED имеет наибольший CV (24.7%) — асинхронное управление окнами создаёт большую «рябь», хотя среднее значение от

этого не страдает.

3.3 Сбросы пакетов

График (рис. 3) представляет моменты сбросов в виде точечной диаграммы: каждая точка — один сброшенный пакет. Три горизонтальные полосы соответствуют трём алгоритмам.

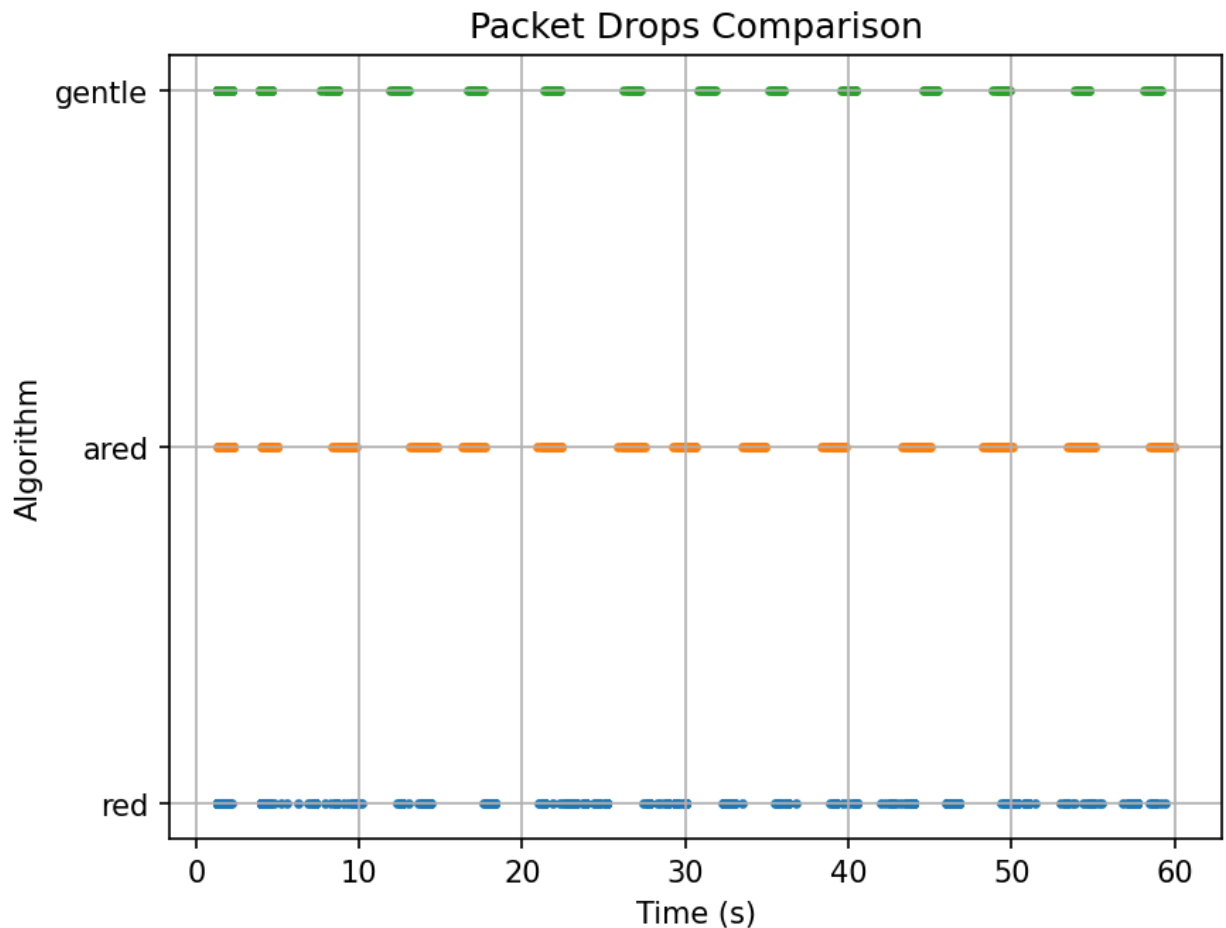


Рис. 3: Моменты сбросов пакетов для каждого алгоритма

RED (нижняя полоса, синий) — самая плотная и непрерывная полоса: точки заполняют практически всё время симуляции, лишь с редкими небольшими просветами. RED активно дропает в 47 из 58 секунд со средним интервалом между сбросами всего **2.3 мс** (медиана). Принудительный сброс при $m_qAvg \geq maxTh$ создаёт непрерывный поток отброшенных пакетов каждый раз, как средняя очередь превышает порог.

ARED (средняя полоса, оранжевый) — чётко видны кластеры сбросов, разделённые заметными паузами. Алгоритм активен в 33 из 58 секунд. Адаптация $maxP$ периодически снижает вероятность сброса до нуля, давая потокам «передышку». Однако когда сбросы происходят, их плотность внутри кластера высока (максимальный всплеск — 276 сбросов в секунду).

Gentle RED (верхняя полоса, зелёный) — наиболее выраженная кластерная структура с самыми длинными паузами между кластерами. Алгоритм активен лишь в 27 из 58 секунд. Внутри каждого кластера сбросы плотные (максимальный всплеск — 305/с), однако между ними — длительные «тихие» промежутки.

Сводная статистика сбросов приведена в табл. 7.

Таблица 7: Статистика сбросов пакетов

Алгоритм	Всего сбросов	Частота (сбр./с)	Медиана интервала	Активных секунд	Макс.
RED	1240	21.34	2.3 мс	47	
ARED	1003	17.11	11.3 мс	33	
Gentle	1026	17.75	9.1 мс	27	

RED генерирует на **19–21% больше сбросов**, чем конкуренты. Медиана интервала между сбросами у RED (2.3 мс) в 4–5 раз меньше, чем у ARED и Gentle, — RED практически непрерывно дропает пакеты в периоды перегрузки, тогда как ARED и Gentle работают «импульсами».

4 Обсуждение

Итоговое сравнение алгоритмов по всем ключевым метрикам представлено в табл. 8.

Таблица 8: Сводное сравнение алгоритмов RED, ARED и Gentle RED

Метрика	RED	ARED	Gentle RED
Средняя пропускная способность	1.8047 Мбит/с	1.8039 Мбит/с	1.7998 Мбит/с
CV пропускной способности	18.7%	21.9%	24.7%
Средняя очередь	7.74 пкт	9.57 пкт	6.40 пкт
Стд. откл. очереди	7.45	11.94	8.86
p99 очереди	26 пкт	49 пкт	34 пкт
Пустая очередь (%)	13.4%	22.3%	26.6%
Всего сбросов	1240	1003	1026
Характер сбросов	непрерывный	кластерный	импульсный
Медиана интервала между сбросами	2.3 мс	11.3 мс	9.1 мс

RED обеспечивает наиболее предсказуемую и стабильную пропускную способность ($CV = 18.7\%$), но достигает этого ценой наибольшего числа сбросов (1240) и наиболее высокой средней очереди. Непрерывный характер сбросов синхронизирует TCP-потoki, создавая жёсткую, «квантованную» динамику управления перегрузкой.

ARED минимизирует число сбросов (1003, -19% к RED) и реже всего тревожит потоки. Однако адаптация $\max P$ создаёт значительную нестабильность очереди (стд. = 11.94, $p99 = 49$ пакетов) с периодами чрезмерного роста буфера — потенциально нежелательными с точки зрения задержки.

Gentle RED достигает наименьшей средней очереди (6.40 пакета) и наибольшей доли пустого буфера (26.6%). Число сбросов сопоставимо с ARED (1026), однако они распределены наиболее компактными импульсами с длинными паузами между ними. Платой является наибольшая вариабельность пропускной способности ($CV = 24.7\%$) вследствие десинхронизации TCP-потоков.

5 Заключение

Проведённый эксперимент показал, что все три алгоритма AQM — RED, ARED и Gentle RED — обеспечивают практически одинаковую среднюю пропускную способность (~ 1.80 Мбит/с, $\sim 90\%$ от номинала канала), однако принципиально различаются по характеру управления буфером и нагрузке на механизм управления перегрузкой TCP.

Ключевое различие состоит в компромиссе между предсказуемостью и агрессивностью управления очередью. RED действует наиболее агрессивно: принудительный сброс при превышении $\max_{Th}$ удерживает очередь «в узде» (средняя 7.74 пакета) и обеспечивает минимальную вариабельность пропускной способности ($CV = 18.7\%$), но ценой наибольшего числа сбросов (1240) и непрерывного давления на TCP-потоки. ARED, напротив, жертвует стабильностью очереди ради снижения числа сбросов: адаптация $\max_R$ уменьшает сбросы на 19% по сравнению с RED, однако порождает широкие осцилляции (std. = 11.94, p99 = 49 пакетов), неприемлемые для приложений, чувствительных к задержке. Gentle RED занимает промежуточное положение, достигая наименьшей средней очереди (6.40 пакета) за счёт более плавной функции сброса, которая десинхронизирует TCP-потоки и снижает заполненность буфера, — но при этом увеличивает вариабельность пропускной способности ($CV = 24.7\%$).

Полученные результаты подтверждают, что выбор алгоритма AQM определяется требованиями конкретного сценария: не существует единственного «лучшего» решения — каждый алгоритм оптимален по своей целевой метрике.

Рекомендации по применению:

- При приоритете **минимальной задержки в буфере** — Gentle RED (наименьшая средняя очередь 6.40 пкт).
- При приоритете **минимального числа сбросов** — ARED (на 19% меньше сбросов, чем RED).
- При приоритете **предсказуемой пропускной способности** — RED (наименьший CV 18.7%).

Список литературы

- [1] Floyd S., Jacobson V. Random Early Detection Gateways for Congestion Avoidance // IEEE/ACM Transactions on Networking. — 1993. — Vol. 1, No. 4. — P. 397–413.
- [2] Floyd S. Recommendation on using the “gentle” variant of RED. — 2000. — URL: <https://www.icir.org/floyd/red/gentle.html>.
- [3] Floyd S., Gummadi R., Shenker S. Adaptive RED: An Algorithm for Increasing the Robustness of RED’s Active Queue Management // Tech. Report, ICSI. — 2001.
- [4] Christiansen M., Jeffay K., Ott D., Smith F.D. Tuning RED for Web Traffic // IEEE/ACM Transactions on Networking. — 2001. — Vol. 9, No. 3. — P. 249–264.
- [5] Gettys J., Nichols K. Bufferbloat: Dark Buffers in the Internet // Communications of the ACM. — 2012. — Vol. 55, No. 1. — P. 57–65.
- [6] Braden B., Clark D., Crowcroft J. et al. Recommendations on Queue Management and Congestion Avoidance in the Internet // RFC 2309, IETF. — 1998.
- [7] Nichols K., Jacobson V., McGregor A., Iyengar J. Controlled Delay Active Queue Management // RFC 8289, IETF. — 2018.
- [8] Nichols K., Jacobson V. Controlling Queue Delay // ACM Queue. — 2012. — Vol. 10, No. 5. — P. 20–34.
- [9] Pan R., Natarajan P., Piglione C. et al. PIE: A Lightweight Control Scheme to Address the Bufferbloat Problem // Proc. IEEE HPSR. — 2013. — P. 148–155.
- [10] Feng W., Kandlur D., Saha D., Shin K. BLUE: A New Class of Active Queue Management Algorithms // Tech. Report CSE-TR-387-99, University of Michigan. — 2002.
- [11] Hollot C.V., Misra V., Towsley D., Gong W. On Designing Improved Controllers for AQM Routers Supporting TCP Flows // Proc. IEEE INFOCOM. — 2001. — P. 1726–1734.
- [12] Королькова А.В. Математическая модель процесса передачи трафика с регулируемой алгоритмом типа RED динамической интенсивностью потока: дис. ... канд. физ.-мат. наук: 05.13.18 / Королькова Анна Владиславовна. — М.: РУДН, 2010. — 115 с.
- [13] Велиева Т.Р. Анализ автоколебательных режимов непрерывно-дискретных динамических моделей: дис. ... канд. физ.-мат. наук: 05.13.18 / Велиева Татьяна Рефатовна. — М.: РУДН, 2019. — 141 с.